

# Labyrinth Composer

## 空间谜题设计工具 UI / UX 设计文档

从“画地图”转向“调试玩家可达性、解锁链和空间体验”的专业创作工具

版本：v0.1

日期：2026-06-15

用途：产品设计、交互设计、原型制作与研发交接

# 文档控制

| 项目   | 说明                                                       |
|------|----------------------------------------------------------|
| 文档名称 | Labyrinth Composer 空间谜题设计工具 UI / UX 设计文档                 |
| 版本   | v0.1                                                     |
| 目标读者 | 产品负责人、UX/UI 设计师、关卡设计师、谜题设计师、技术负责人、客户端开发、引擎集成开发           |
| 文档目标 | 定义软件的信息架构、主工作区、核心视图、关键交互、用户流程、设计系统、MVP 范围和体验验收标准。        |
| 设计前提 | 软件服务于传统迷宫、Zelda 式地牢、恐怖解谜、叙事迷宫、Metroidvania、密室逃脱等空间类谜题设计。 |

## 核心 UX 命题

Labyrinth Composer 的用户不是在单纯画一张地图，而是在设计并验证玩家如何探索、受阻、获得解法、回溯、解锁，并重新认识空间。

# 目录

- 1. 产品 UX 总目标
- 2. 用户角色与核心场景
- 3. 信息架构
- 4. 主界面布局
- 5. 核心对象与视觉语义
- 6. 五个核心视图
- 7. 完整用户流程
- 8. 关键任务流程
- 9. 设计系统
- 10. 交互模型
- 11. 新手体验与专家体验
- 12. 验证中心 UX
- 13. 自动生成器 UX
- 14. Playtest Review 测试回放
- 15. 导出与集成 UX
- 16. MVP UI/UX 范围
- 17. 体验验收标准
- 18. 附录：Wireframe 文案与组件清单

# 1. 产品 UX 总目标

Labyrinth Composer 的定位是“空间谜题编排器”：它介于关卡编辑器、谜题依赖图编辑器、可达性模拟器和体验时间线工具之间。它优先解决空间类游戏中最难通过普通地图编辑器发现的问题：玩家现在能去哪、为什么不能去、解法在哪里、是否会死锁、回溯是否有意义、线索是否足够，以及玩家是否能理解下一步目标。

| 目标       | 用户获得的价值                             | 界面体现                                          |
|----------|-------------------------------------|-----------------------------------------------|
| 快速建立逻辑骨架 | 用户先设计空间进度、锁与解法，而不是陷入地图细节。           | Progression View、从目标反向生成、模板向导                 |
| 实时发现设计错误 | 像 IDE 检查代码一样检查死锁、循环依赖、软锁、提示不足和回溯过长。 | 顶部健康状态、底部 Problems、Validation Center          |
| 把结构转化为体验 | 帮助用户评估目标感、回溯张力、恐怖强度、叙事揭示和奖励密度。      | Timeline View、Spatial Overlay、Playtest Review |

## 一句话定义

UI 应该像 Figma 一样自由编辑，像 IDE 一样实时检查，像导演时间线一样分析体验，像游戏调试器一样模拟玩家状态。

# 2. 用户角色与核心场景

同一套工具需要服务多种设计岗位，但不同角色关注点不同。产品界面应允许用户从“视图”和“模板”两个维度进入，而不是强迫所有用户使用同一条路径。

| 用户类型            | 主要关注                          | 最重要视图 / 功能                                                             |
|-----------------|-------------------------------|------------------------------------------------------------------------|
| 关卡设计师           | 空间结构、主线路径、锁钥关系、回溯成本、可通关性。     | Progression View、Spatial View、Reachability Simulator、Validation Center |
| 谜题设计师           | 输入输出、线索分布、知识锁、多步骤谜题链、循环依赖。    | Dependency View、条件构建器、线索追踪、知识状态模拟                                      |
| 叙事设计师           | 玩家什么时候知道什么、真相揭示顺序、空间重解释、情绪节奏。 | Timeline View、Knowledge Token、Narrative Fact、叙事揭示曲线                    |
| 独立开发者 / 生成系统开发者 | 快速搭建、模板、自动生成、JSON 导出、引擎集成。    | 模板库、Generator、DSL / JSON 编辑器、Export / Integration                      |

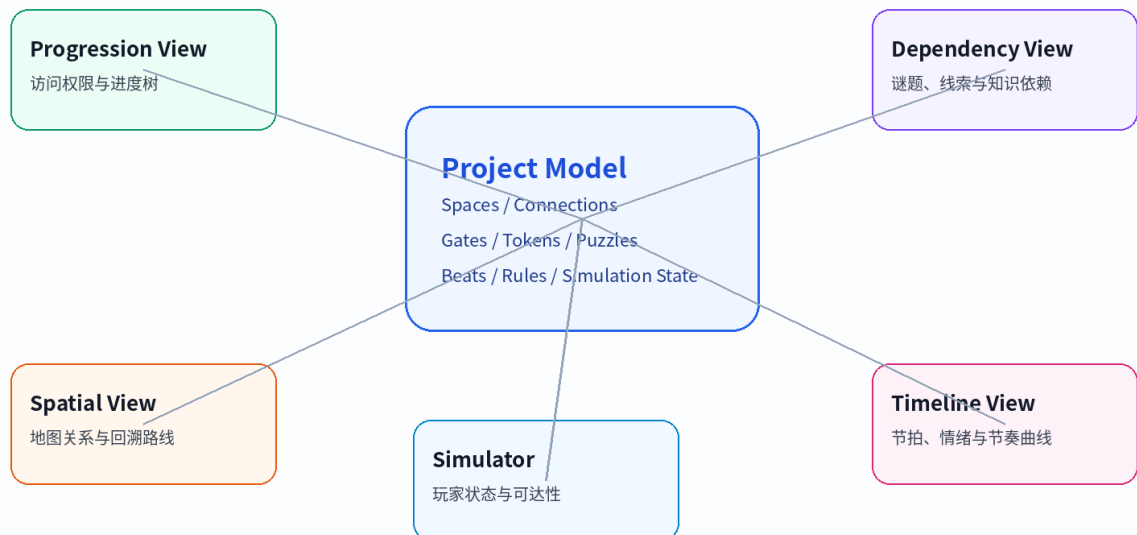
## 2.1 典型使用场景

- 传统迷宫：分析死路、环路、路径选择、地标分布和错误路径奖励。
- Zelda 式地牢：设计小钥匙、Boss Key、中心大厅、道具能力门和新能力回收点。
- 恐怖解谜：设计逃生门、保险丝、配电室、安全屋、追逐路线和回程事件。
- 叙事迷宫：设计知识锁、记忆碎片、门牌含义、真相揭示顺序和旧空间重解释。
- Metroidvania：设计能力获得、旧区域重访、单向落差、软锁检测和捷径。

### 3. 信息架构

软件主结构不应拆成许多孤立页面，而应该围绕同一份 Project Model 提供多种视图投影。用户在任何视图中选择对象，其他视图同步高亮。

信息架构：一份数据，多种视图投影



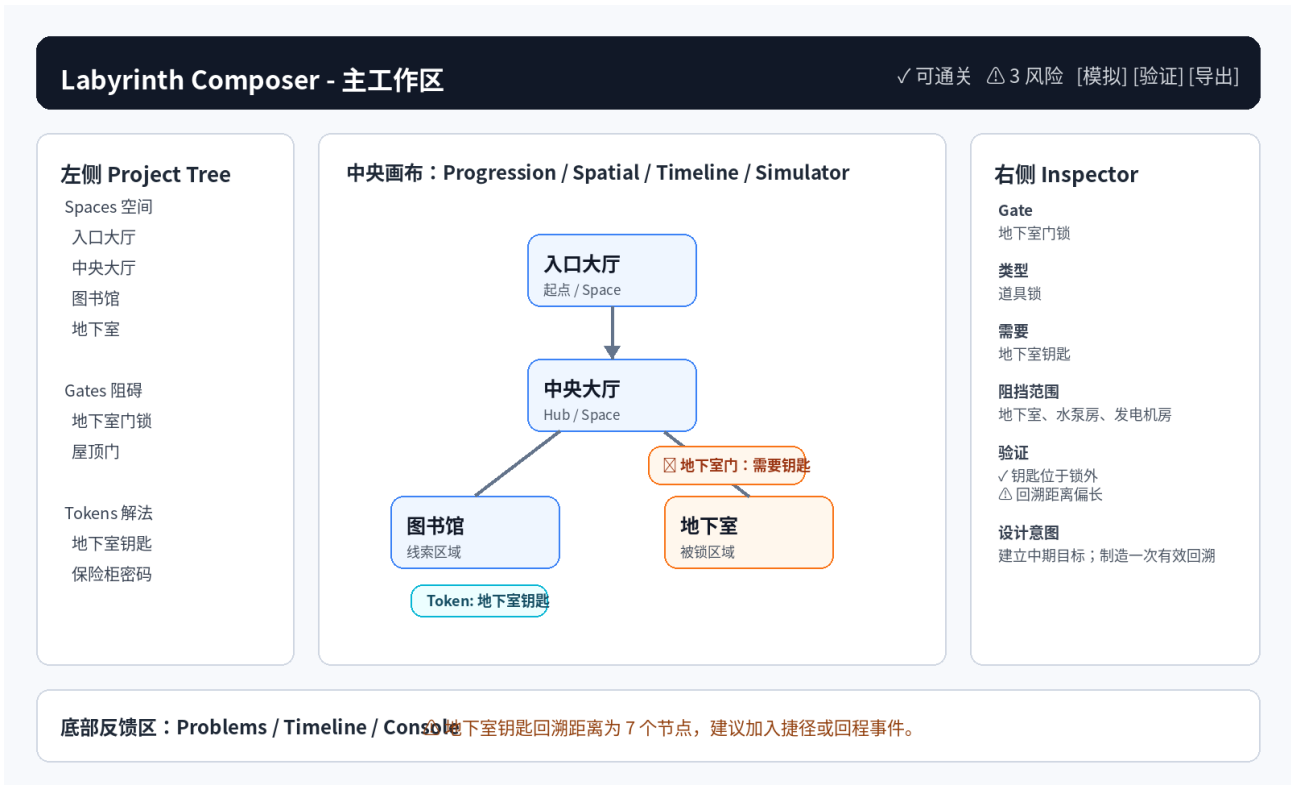
所有视图都读写同一份项目数据：在一个视图选择 Gate，其他视图同步高亮对应空间、依赖和体验节拍。

图 1：信息架构 - 一份项目数据，多种视图投影

| 一级区域                 | 用途                     | 关键对象                                              |
|----------------------|------------------------|---------------------------------------------------|
| Project Dashboard    | 新建项目、选择模板、打开最近项目、管理版本。 | Project、Template、Recent File                      |
| Workspace 主工作区       | 日常设计的核心区域，承载多视图编辑。     | Space、Gate、Token、Puzzle、Beat                      |
| Pattern Library      | 提供空间谜题语法和类型模板。         | Lock-Key、Ability-Gate、Knowledge-Gate、State-Change |
| Validation Center    | 集中展示设计健康度、错误链和修复建议。    | Rule、Issue、Fix Action                             |
| Generator            | 基于设计意图自动生成主线、支线或事件骨架。  | Prompt、Constraint、Candidate Plan                  |
| Playtest Review      | 导入真实玩家数据并叠加在设计图上。      | Path、Heatmap、Attempt、Death、Dwell Time             |
| Export / Integration | 输出设计文档、项目数据和引擎集成格式。    | DOCX、PDF、JSON、Unity、Unreal、Godot                  |

4. 主界面布局

主工作区采用专业创作工具常见的“四区结构”：顶部工具栏、左侧对象导航、中间画布、右侧属性面板、底部反馈面板。这样可以同时满足快速编辑、局部精修和持续验证。



|            |                         |                |
|------------|-------------------------|----------------|
|            | 合、机关。                   |                |
| Beats 体验节拍 | 探索、阻碍、线索、奖励、回溯、恐怖、叙事揭示。 | 高强度、长回溯、连续同质节拍 |

### 4.3 右侧 Inspector

Inspector 是精细编辑区。选择不同对象时显示不同属性，但应保持统一结构：基础信息、功能角色、依赖关系、体验属性、验证结果、修复建议。

| 选中对象       | Inspector 应显示                        |
|------------|--------------------------------------|
| Space      | 名称、类型、角色、所属区域、是否主线、放置内容、情绪强度、预计停留时间。 |
| Connection | 连接类型、是否单向、是否隐藏、是否有 Gate、锁定范围、回溯距离。   |
| Gate       | 阻碍类型、需要条件、可见性、阻挡范围、锁前提示、验证结果。        |
| Token      | 类型、获得位置、用途、获得条件、回溯距离、是否可能遗忘。         |
| Puzzle     | 输入、输出、位置、线索来源、解法验证、是否可跳过。            |
| Beat       | 类型、空间、情绪、强度、目标、触发条件、持续时间。            |

### 4.4 底部反馈区

底部面板相当于 IDE 的 Problems / Console。任何错误都必须可点击，并能在画布中高亮相关链路。问题卡片应包含“是什么、为什么、影响什么、如何修复”。

|                                |
|--------------------------------|
| Problems: 0 errors, 3 warnings |
| ⚠ 回溯过长：从地下墓穴返回屋顶入口需要 14 个节点。   |
| ⚠ 提示不足：保险柜密码没有明确线索来源。          |
| ⚠ 节奏单调：连续 6 个节点都是探索，没有奖励或阻碍。   |

## 5. 核心对象与视觉语义

软件的基本对象需要足够抽象，才能覆盖不同类型的空间谜题。UI 不能只假设“钥匙开门”，而要把钥匙扩展为物品、知识、状态、能力、关系和叙事理解。

| 对象         | 定义                      | 视觉建议         | 典型例子                  |
|------------|-------------------------|--------------|-----------------------|
| Space      | 玩家可以访问或理解的一片空间，可物理也可叙事。 | 圆角矩形。        | 大厅、图书馆、地下室、记忆场景、梦境片段  |
| Connection | 两个空间之间的通路或跳转。           | 线 / 箭头。      | 门、楼梯、暗道、电梯、剧情切换       |
| Gate       | 阻断访问或提高访问代价的条件。         | 锁形标签，贴在连接线上。 | 钥匙门、密码门、黑暗走廊、Boss、毒气区 |

|        |                 |             |                        |
|--------|-----------------|-------------|------------------------|
| Token  | 玩家获得后能改变可达性的解法。 | 胶囊标签。       | 钥匙、密码、手电筒、二段跳、0314 的含义 |
| Puzzle | 输入转输出的谜题单元。     | 菱形 / 六边形。   | 保险柜、雕像朝向、发电机、三线索推理     |
| Beat   | 玩家体验中的一个节拍。     | 时间线圆点 / 卡片。 | 首次看见门、获得钥匙、回程怪物出现      |

## 5.1 色彩语义

颜色必须表达状态，不能只作为装饰。同一种颜色在全软件中应始终表达同一种含义。

| 颜色 | 语义                 |
|----|--------------------|
| 蓝色 | 可达空间 / 当前阶段可访问内容   |
| 红色 | 严重错误 / 死锁 / 主线不可通关 |
| 黄色 | 风险 / 需要设计师注意       |
| 绿色 | 已满足条件 / 验证通过 / 可通关 |
| 紫色 | 知识、叙事事实、理解状态       |
| 橙色 | Gate / 阻碍 / 待解锁通道  |
| 青色 | Token / 可获得解法      |
| 灰色 | 未知、未访问、未定义、低优先级    |

### 重要限制

红色应保留给验证错误，不应同时拿来表示恐怖强度。恐怖强度可使用单独的 Heat Overlay，避免和错误状态冲突。

## 6. 五个核心视图

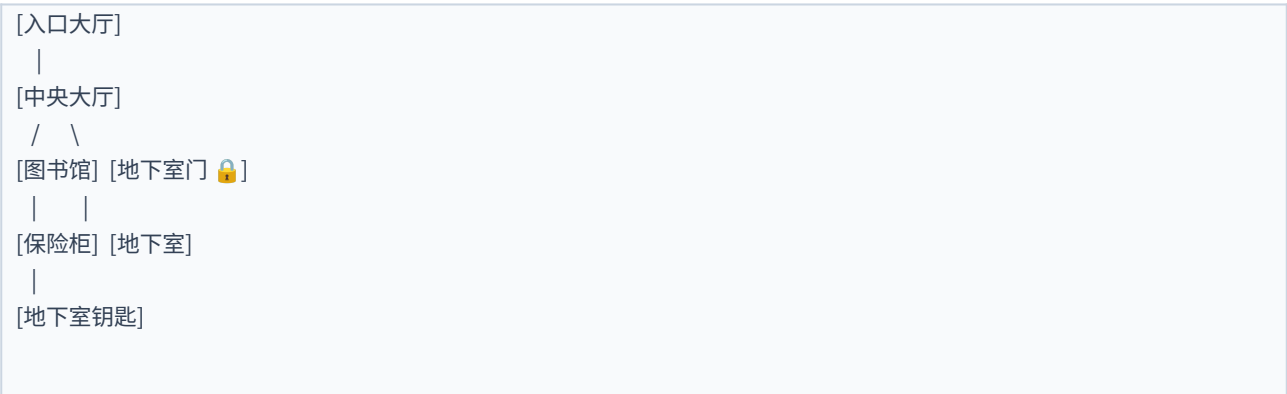
五个核心视图是同一份关卡数据的不同观察方式。用户不应在不同视图中重复录入信息，而应在一个视图里修改对象，其他视图自动同步。

### 6.1 Progression View：进度结构视图

默认视图。它展示玩家访问权限如何展开，是 Environment Tree 理论在工具中的直接落地。设计师在此创建空间、连接、Gate、Token 和 Puzzle，并实时查看锁定子树。

| 能力        | 说明                              |
|-----------|---------------------------------|
| 锁定子树高亮    | 在连接线上添加 Gate 后，高亮所有被阻断的后续空间。    |
| 合法放置区     | 创建 Token 时显示哪些空间可以放置，哪些空间会导致死锁。 |
| 主线 / 支线区分 | 主线路径用粗线，支线用细线，可选奖励用虚线。          |
| 错误链定位     | 点击错误后高亮 Gate、Token、被锁区域和依赖路径。   |

|      |                  |
|------|------------------|
| 折叠区域 | 大型关卡可折叠楼层、区域或章节。 |
|------|------------------|



## 6.2 Spatial View：空间布局视图

Spatial View 负责空间感、几何关系和回溯路线。它可以先做节点图，后续扩展到网格地图或 3D Blockout。

| 模式          | 适用情况                               | 关键 Overlay         |
|-------------|------------------------------------|--------------------|
| 节点图模式       | 早期概念设计、房间关系梳理。                     | 主线路径、锁定区域、回溯路线     |
| 网格模式        | 传统迷宫、俯视角地牢、Roguelike。              | 死路密度、地标分布、错误路径奖励   |
| Blockout 模式 | 需要导入 Unity / Unreal / Godot 的关卡原型。 | 门位置、高度差、视线、捷径、追逐路线 |

## 6.3 Dependency View：谜题依赖图

Dependency View 专门展示线索、知识、谜题、Token、Gate 和新空间之间的依赖。它是叙事迷宫、密室逃脱和知识锁设计的关键视图。

[病房记录] —┐

[墙上时钟] —┴—> [保险柜密码] —> [保险柜谜题] —> [地下室钥匙]

[儿童涂鸦] —┐

| 检查项      | 设计意义                      |
|----------|---------------------------|
| 必要线索是否可达 | 防止关键线索被放在谜题之后。            |
| 线索是否过少   | 避免玩家无法合理推出答案。             |
| 线索是否过早   | 避免关键答案在目标出现前失去意义。         |
| 是否存在循环依赖 | 检测 A 需要 B、B 需要 C、C 又需要 A。 |
| 是否存在单点失败 | 某个关键线索错过后是否无法通关。          |

## 6.4 Timeline View：体验时间线

Timeline View 检查“过程是否有张力”，而不只是“逻辑是否可通关”。每个 Beat 记录空间、体验类型、情绪、强度、目标清晰度和预计持续时间。



| 探索 → 阻碍 → 线索 → 谜题 → 奖励 → 回溯 → 恐惧事件 → 解锁 → 新空间 |                   |
|-----------------------------------------------|-------------------|
| 曲线                                            | 用于发现的问题           |
| 探索密度                                          | 是否过长时间没有新目标或新信息。  |
| 谜题密度                                          | 是否连续谜题过多，缺少探索和奖励。 |
| 恐怖强度                                          | 是否高压持续过久，缺少安全缓冲。  |
| 叙事揭示强度                                        | 真相是否过早、过晚或过密集。    |
| 回溯成本                                          | 是否存在无意义长回溯。       |
| 目标清晰度                                         | 玩家是否知道为什么要去下一个空间。 |

### 6.5 Reachability Simulator：可达性模拟器

模拟器让设计师像调试程序一样调试玩家状态。用户可以拖动阶段滑块，查看每一阶段玩家的位置、拥有物品、已知信息、世界状态、可达空间和不可达原因。



图 3：可达性模拟器 Wireframe

#### 核心体验

点击不可达区域时，系统必须解释“为什么不可达”，并显示完整因果链，而不是只显示“不可达”。

# 7. 完整用户流程

## 核心用户流程：从目标反向设计空间谜题



体验目标：用户不是在“画地图”，而是在调试玩家如何探索、受阻、获得解法并重新认识空间。

图 4：从目标反向设计的核心用户流程

## 7.1 首次进入软件

首次进入不应直接给用户一张空白画布。应先让用户选择设计类型，因为不同类型决定默认对象、检查规则、模板和体验曲线。

| 模板入口         | 默认启用的设计语法                           |
|--------------|-------------------------------------|
| 传统迷宫         | 路径选择、死路、环路、地标、错误路径奖励。               |
| Zelda 式地牢    | 小钥匙、Boss Key、中心大厅、Dungeon Item、能力门。 |
| 恐怖解谜         | 逃生目标、资源门、危险回溯、安全屋、追逐路线。             |
| 叙事迷宫         | 知识锁、记忆碎片、叙事事实、空间重解释。                |
| Metroidvania | 能力解锁、旧区域重访、单向落差、软锁检测。               |
| 密室逃脱         | 线索链、组合谜题、信息整合、局部空间高密度交互。            |

## 7.2 新建项目向导

- Step 1：输入项目名称、关卡目标、预计时长、玩家视角和核心体验。
- Step 2：选择空间结构模板，如中心大厅、多层建筑、线性深入、环形回路。
- Step 3：定义最终目标被什么阻碍，如电力未恢复、钥匙、密码、Boss 或叙事理解。
- Step 4：从最终目标反向生成谜题链，并由系统提示合法解法放置区。
- Step 5：进入主工作区，用户看到一个可编辑、可验证的初始骨架。

## 7.3 日常设计循环

建立空间 → 添加阻碍 → 放置解法 → 验证可达性 → 模拟玩家 → 调整体验 → 导出

这个循环应尽量低摩擦。用户在任意一步都可以运行验证，也可以从验证问题直接跳回编辑对象。

## 8. 关键任务流程

### 8.1 从终点反向设计主线

| 步骤       | 界面行为                                | 系统反馈               |
|----------|-------------------------------------|--------------------|
| 选择目标空间   | 用户点击“屋顶小屋”或“正门”。                    | 右侧显示目标属性和当前访问条件。   |
| 添加 Gate  | 点击“保护这个目标”。选择钥匙门、密码门、能力门、状态门、叙事理解等。 | 系统创建 Gate 并计算锁定子树。 |
| 创建 Token | 系统自动创建所需解法，例如“屋顶钥匙”。                | 显示合法放置区与非法区域。      |
| 放置 Token | 用户把 Token 放到空间、Puzzle 或 NPC 上。      | 若导致死锁，立即提示并给修复方案。  |
| 继续反向生成   | 如果 Token 所在区域也被锁，继续创建前置谜题。          | 形成主线谜题链并实时验证。      |

### 8.2 修复死锁

当系统检测到“钥匙在自己解锁的门后面”这类问题时，应提供可执行修复，而不只是报警。

严重错误：地下室钥匙位于地下室内部。  
原因链：地下室门 → 需要地下室钥匙 → 地下室钥匙位于地下室。  
修复建议：  
1. 移动地下室钥匙到图书馆。  
2. 改为密码门，密码线索放在图书馆。  
3. 添加远程获取机制，如从门缝用磁铁取得钥匙。

### 8.3 设计 Zelda 式道具回收

用户获得新能力后，系统应检查它是否充分改变旧空间。例如获得“钩爪”后，至少应出现若干新可达路径、支线奖励或主线用途。

| 指标        | 推荐判断              |
|-----------|-------------------|
| 获得前可见使用点  | 至少 1-2 个，用于建立期待。  |
| 获得后新增可达空间 | 主线至少 1 个，支线可多个。   |
| 能力使用次数    | 过少会降低能力价值感。       |
| 旧区域重解释    | 最好有已访问空间因新能力变得不同。 |

### 8.4 设计恐怖回程

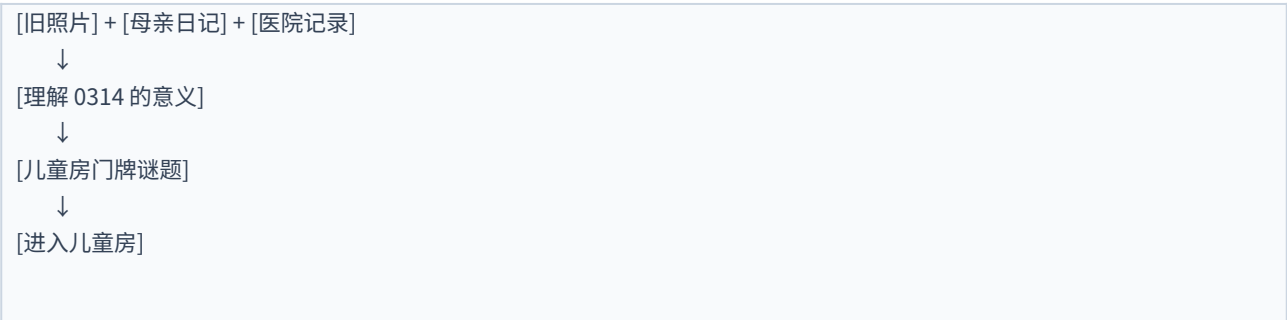
恐怖解谜的回溯常常不是缺点，而是张力来源。系统不应简单批评“回溯过长”，而应结合游戏类型给出情绪化建议。

| 触发条件 | 建议事件 |
|------|------|
|------|------|

|                     |                             |
|---------------------|-----------------------------|
| 关键 Token 获得后需要返回旧区域 | 灯光故障、怪物首次出现、走廊变化、远处哭声。      |
| 回程路径过长但无变化          | 添加捷径、追逐、环境改变或新的叙事信息。        |
| 高压段缺少缓冲             | 添加安全屋、藏身点、安静房间或资源补给。        |
| 追逐路线不成立             | 检查玩家路径、怪物路径、门宽、转角、藏身点和脚本延迟。 |

## 8.5 设计叙事知识锁

知识锁要求系统模拟玩家“知道什么”，而不是只模拟玩家“拥有什么”。例如 0314 门牌谜题需要旧照片、母亲日记和医院记录共同生成“理解 0314 的意义”这个 Knowledge Token。



# 9. 设计系统

视觉风格应偏专业创作工具：清晰、克制、结构感强、支持高信息密度和长时间工作。推荐默认深色主题，同时提供浅色主题。

## 9.1 组件层级

| 组件                | 作用                                   | 关键状态                                       |
|-------------------|--------------------------------------|--------------------------------------------|
| Node Card         | 表示 Space、Puzzle、Clue、Beat 等实体。       | 可达、锁定、主线、支线、错误、风险、未定义。                     |
| Gate Label        | 贴在 Connection 上，表示阻碍。                | 已满足、缺条件、隐藏、未提示、严重错误。                       |
| Token Chip        | 表示物品、知识、能力或状态。                       | 未放置、可获得、已获得、未使用、用途过远。                      |
| Problem Card      | 说明设计问题和修复方案。                         | Error、Warning、Suggestion、Resolved、Ignored。 |
| Overlay Toggle    | 切换主线路径、锁定区域、回溯、威胁、线索等叠加层。            | 开 / 关 / 聚焦。                                |
| Condition Builder | 配置 Gate 或 Puzzle 的 AND / OR / 自定义条件。 | 有效、循环、引用缺失、复杂条件。                           |

## 9.2 条件构建器

Gate 条件不能只是一行文本。应支持 AND、OR、条件组和自定义表达式，以覆盖复杂谜题。

打开墓穴门需要：  
(银钥匙 OR 铜钥匙)  
AND 知道墓穴铭文含义  
AND 水位已下降

### 9.3 节点卡片示例

Space 节点

图书馆

Space / Clue Area

Token: 1 Puzzle: 1

提示不足

Gate 标签

地下室门：需要地下室钥匙

Puzzle 节点

保险柜谜题

输入：密码线索

输出：地下室钥匙

## 10. 交互模型

### 10.1 拖拽创建

拖拽是主要编辑方式。对象之间的拖拽应自动建立语义关系，减少表单输入。

| 操作                     | 自动结果                   |
|------------------------|------------------------|
| 把 Gate 拖到 Connection 上 | Gate 成为该连接的阻碍，并计算锁定子树。 |
| 把 Token 拖到 Space 上     | 设置 Token 的 location。   |
| 把 Token 拖到 Gate 上      | 设置 Gate 的 requires。    |
| 把 Clue 拖到 Puzzle 上     | 设置 Puzzle 的输入。         |
| 把 Puzzle 拖到 Space 上    | 设置 Puzzle 的发生位置。       |
| 把 Beat 拖到 Timeline 上   | 创建体验节拍并绑定空间。           |

### 10.2 右键菜单

| 上下文        | 菜单项                                            |
|------------|------------------------------------------------|
| Space      | 添加子空间、添加连接、放置 Token、添加 Puzzle、设为起点、设为目标、查看可达性。 |
| Connection | 添加 Gate、设为单向、设为捷径、设为隐藏通道、设为剧情通道。               |
| Gate       | 创建所需 Token、显示锁定区域、显示合法解法位置、转                   |

|              |                                        |
|--------------|----------------------------------------|
|              | 换为知识锁 / 状态锁。                           |
| Token        | 显示所有用途、移动到合法区域、转换为知识 / 能力 / 状态、设为可选奖励。 |
| Problem Card | 定位对象、显示因果链、一键修复、忽略本规则、创建 TODO。         |

## 10.3 命令面板与快捷键

专业用户需要低摩擦操作。建议提供 Command Palette，并保留常用快捷键。

| 快捷键            | 动作         |
|----------------|------------|
| Cmd / Ctrl + K | 打开命令面板。    |
| S              | 创建 Space。  |
| G              | 创建 Gate。   |
| T              | 创建 Token。  |
| P              | 创建 Puzzle。 |
| B              | 创建 Beat。   |
| C              | 连接两个节点。    |
| R              | 运行模拟。      |
| F              | 聚焦当前问题。    |
| Tab            | 在核心视图间切换。  |

# 11. 新手体验与专家体验

## 11.1 新手体验

新手体验的目标是在 10 分钟内让用户理解 Space、Gate、Token、Puzzle、Simulator 和 Validation 的关系。

- 用模板代替空白画布：默认提供钥匙门迷宫、Zelda 中央大厅、恐怖逃生公寓、叙事老宅等模板。
- 用真实操作做教程：创建目标、添加 Gate、创建 Token、放置 Token、运行模拟。
- 术语分层：新手模式显示“区域 / 阻碍 / 解法 / 谜题 / 体验节拍”，专家模式显示 Space / Gate / Token / Puzzle / Beat。
- 设计意图提示：创建 Gate 时询问它是为了控制进度、制造回溯、建立目标、制造恐惧还是叙事揭示。

## 11.2 专家体验

专家用户需要速度、可控性和可扩展性。

| 能力    | 价值                        |
|-------|---------------------------|
| 高级图编辑 | 多选、对齐、自动布局、图层、折叠子树、节点分组、注 |

|                |                                    |
|----------------|------------------------------------|
|                | 释框、书签。                             |
| DSL / JSON 编辑器 | 文本模式直接编辑项目数据，适合程序化生成和版本控制。         |
| 规则自定义          | 按项目类型定义回溯阈值、提示要求、能力使用次数、恐怖强度上限等。   |
| 版本对比           | 比较两个版本的空间结构、依赖关系、验证结果和体验曲线变化。      |
| 协作评论           | 在节点或问题卡片上 @ 团队成员，形成关卡设计 Review 流程。 |

## 12. 验证中心 UX

Validation Center 应像“设计健康报告”，而不是普通错误列表。它既要指出问题，也要解释原因和提供修复动作。

| 维度     | 检查内容                                  |
|--------|---------------------------------------|
| 可通关性   | 目标是否可达，主线是否完整。                        |
| 死锁     | 关键 Token 是否位于自己解锁的区域之后。               |
| 循环依赖   | A 需要 B，B 需要 C，C 又需要 A。                |
| 软锁     | 单向门、资源消耗、不可逆状态是否可能导致无法通关。             |
| 提示与线索  | 关键 Gate 前是否有足够提示，知识锁是否有可达线索。          |
| 回溯     | 获得 Token 到使用 Gate 的空间距离是否过长，回程是否有变化。  |
| 节奏     | 是否连续同质节拍、奖励不足、恐怖强度过高或目标感过弱。           |
| 类型专属规则 | Zelda 小钥匙顺序、恐怖安全屋、Metroidvania 能力回收等。 |

### 12.1 问题卡片结构

|                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>严重错误：屋顶钥匙不可获得</p> <p>原因：<br/>屋顶钥匙位于屋顶小屋。<br/>但进入屋顶小屋需要屋顶钥匙。</p> <p>影响：<br/>玩家无法进入屋顶小屋，主线不可通关。</p> <p>建议修复：<br/>[移动屋顶钥匙到地下室] [改为屋顶密码] [添加远程拾取机制] [标记为非主线]</p> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|

验证器的 UX 原则

每个问题都应包含四件事：是什么问题、为什么是问题、影响什么、如何修复。

13. 自动生成器 UX

Generator 不应被设计成“随机地图按钮”，而应是设计助手。它根据用户意图生成可验证的结构候选，并允许用户比较、预览和局部应用。

| 生成类型        | 输入参数                            | 输出                                   |
|-------------|---------------------------------|--------------------------------------|
| 主线谜题链       | 目标、长度、谜题数量、锁类型比例。               | Gate / Token / Puzzle 依赖链。           |
| 支线奖励区       | 分支数量、奖励类型、回溯容忍度。                | 可选空间、奖励 Token、提示路径。                  |
| Zelda 小钥匙结构 | 钥匙数量、锁数量、Boss Key、Dungeon Item。 | 可验证的钥匙门进度树。                          |
| 恐怖回溯事件      | 回溯长度、威胁数量、安全屋、追逐次数。             | 事件 Beat、威胁路径、逃生路线。                   |
| 叙事线索链       | 真相节点、线索数量、误导线索、揭示顺序。            | Clue / Knowledge / Narrative Fact 图。 |

13.1 生成结果预览

生成器应给出多个候选，而不是直接改写项目。用户可以比较方案 A、B、C 的可通关性、回溯距离、谜题数量、空间复杂度和体验曲线，再选择应用。

14. Playtest Review 测试回放

成熟版本应支持导入真实玩家测试数据，将“设计预期”和“玩家实际行为”叠加比较。

| 导入数据      | 叠加视图              | 可发现问题             |
|-----------|-------------------|-------------------|
| 玩家路径      | Spatial View 热力图。 | 玩家是否迷路、是否绕开主线。    |
| 停留时间      | 空间节点热度。           | 哪里理解成本过高或缺少目标。    |
| 门锁尝试次数    | Gate 问题卡。         | 玩家是否拿到解法后不知道去哪里用。 |
| 谜题完成时间    | Dependency View。  | 线索是否不足或过度隐蔽。      |
| 死亡点 / 放弃点 | 威胁路径和时间线。         | 追逐是否不公平，压力是否过高。   |
| 未访问支线     | Progression View。 | 支线入口是否缺少诱因或地标。    |

设计预期：玩家获得地下室钥匙后返回中央大厅。  
真实测试：70% 玩家去了二楼走廊；45% 玩家没有意识到地下室门可以打开。  
可能原因：门的视觉提示不足；钥匙描述没有指向地下室；从图书馆到地下室门缺少地标引导。



# 15. 导出与集成 UX

导出功能应服务不同接收者。用户选择“导出给谁”，系统自动调整输出内容重点。

| 接收者    | 导出重点                        | 格式                                                                |
|--------|-----------------------------|-------------------------------------------------------------------|
| 程序员    | ID、类型、连接关系、条件表达式、状态变化、验证规则。 | JSON、YAML、Unity ScriptableObject、Unreal Data Asset、Godot Resource |
| 关卡设计团队 | 空间图、进度树、锁定区域、回溯路线、问题清单。     | PDF、DOCX、SVG、PNG                                                  |
| 叙事团队   | 线索顺序、知识状态、叙事实实、揭示节拍。        | DOCX、PDF、CSV                                                      |
| 制作人    | 关卡目标、主线流程、体验节奏、风险列表、制作量估算。  | PDF、PPTX 摘要                                                       |
| 外包团队   | 空间清单、节点说明、交互要求、验收标准。        | DOCX、PDF、表格                                                       |

## 15.1 导出内容清单

- 设计文档：关卡目标、核心结构、主线路径、谜题依赖、体验时间线、风险清单。
- 项目数据：spaces、connections、gates、tokens、puzzles、beats、rules、metadata。
- 图像输出：进度树、依赖图、空间图、回溯路线、体验曲线。
- 测试清单：需要重点 Playtest 的锁、谜题、回溯和叙事节点。

# 16. MVP UI/UX 范围

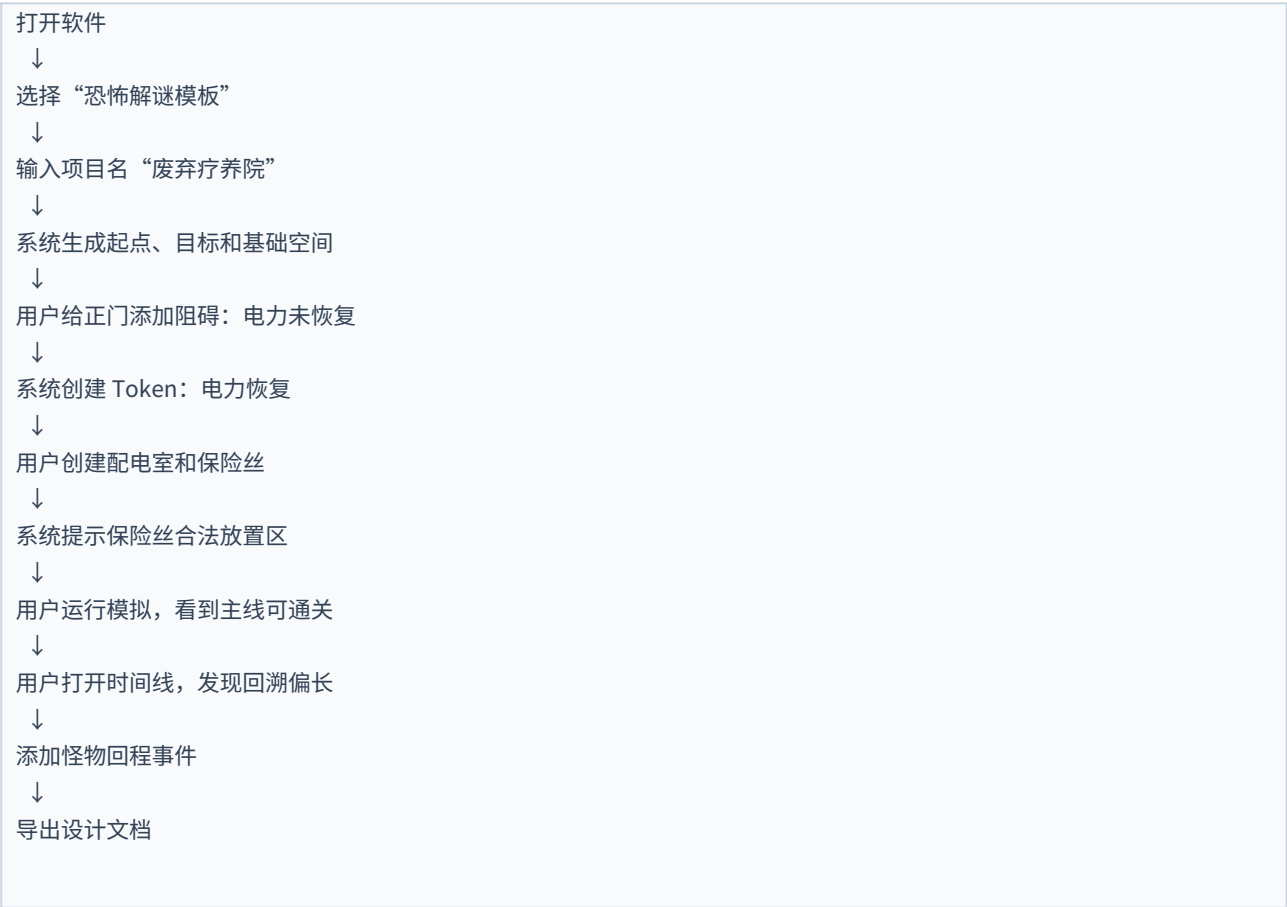
MVP 的目标不是完成一个全功能关卡编辑器，而是验证核心价值：用户能设计一个空间谜题结构，并知道它是否会死锁。

| 必须包含                   | 说明                                     |
|------------------------|----------------------------------------|
| 项目首页                   | 新建项目、选择模板、打开最近项目。                      |
| 新建项目向导                 | 选择类型、目标、空间模板、核心阻碍。                     |
| Progression View       | 创建 Space、Connection、Gate、Token、Puzzle。 |
| 基础 Spatial View        | 节点图模式即可，不要求 3D Blockout。               |
| 右侧 Inspector           | 编辑选中对象属性。                              |
| 底部 Validation Panel    | 显示错误、风险、建议。                            |
| Reachability Simulator | 显示可达空间、不可达原因和阶段滑块。                     |
| 模板库                    | 至少覆盖传统迷宫、Zelda、恐怖解谜、叙事迷宫。              |
| JSON 导出                | 支持后续引擎和研发管线。                           |
| 设计文档导出                 | 输出 DOCX / PDF 摘要。                      |

| 可以暂缓 | 原因 |
|------|----|
|------|----|

|                     |                        |
|---------------------|------------------------|
| 实时多人协作              | 复杂度较高，不影响核心验证。         |
| Playtest 数据回放       | 需要游戏端埋点配合，可作为二期功能。     |
| 复杂 3D Blockout      | 容易变成完整关卡编辑器，偏离 MVP。    |
| Unity / Unreal 深度插件 | 可先用 JSON 输出验证需求。       |
| AI 自动生成完整关卡         | 先做好结构生成和验证，再扩展 AI 生成。  |
| 移动端                 | 复杂图编辑不适合手机，后续做查看与评论即可。 |

## 16.1 MVP 首次用户流程



## 17. 体验验收标准

以下标准用于判断 UI / UX 是否真正支撑了产品目标。

| 验收问题                     | 通过标准                                                                |
|--------------------------|---------------------------------------------------------------------|
| 用户是否能在 10 分钟内理解核心概念？     | 理解 Space 是空间，Gate 是阻碍，Token 是解法，Simulator 显示玩家能去哪，Validation 告诉哪里错。 |
| 用户是否能在 30 分钟内设计一个可通关小关卡？ | 能完成 6-10 个空间、2-3 个 Gate、2 个 Puzzle、1 条主线、1 条支线，并通过验证。               |
| 用户是否能一眼看懂死锁原因？           | 错误卡能显示完整因果链，并能高亮图中相关节点。                                             |
| 工具是否比纸笔更适合早期关卡设计？        | 创建锁、放置解法、查看合法区域和运行模拟的成本足够低。                                         |

|                  |                                                                  |
|------------------|------------------------------------------------------------------|
| 工具是否能服务中后期制作？    | 提供验证、模拟、导出、协作或测试数据回放能力。                                          |
| 不同游戏类型是否能共用同一模型？ | 传统迷宫、Zelda、恐怖解谜和叙事迷宫均可用 Space + Gate + Token + Puzzle + Beat 表达。 |

### MVP 成功指标

用户能用它比纸笔更快地搭出一条空间谜题主线，并通过模拟器看清楚玩家为什么能或不能到达目标。

## 18. 附录：Wireframe 文案与组件清单

### 18.1 项目首页文案

欢迎使用 Labyrinth Composer

你想设计什么类型的空间谜题？

[传统迷宫] [Zelda 式地牢] [恐怖解谜] [叙事迷宫] [Metroidvania] [密室逃脱] [从空白开始]

Zelda 式地牢：适合锁、钥匙、道具、Boss Key、中心大厅与回溯结构。

恐怖解谜：适合逃生目标、资源门、危险回溯、安全屋和追逐路线。

叙事迷宫：适合知识锁、记忆碎片、真相揭示和空间重解释。

### 18.2 空状态文案

| 状态    | 建议文案 / 操作                                         |
|-------|---------------------------------------------------|
| 空白项目  | 你可以从这里开始：[创建起点] [创建目标] [从目标反向生成谜题链] [选择模板] [导入地图] |
| 无验证结果 | 还没有运行验证。点击“验证”后，系统会检查可通关性、死锁、回溯和提示问题。             |
| 图太复杂  | 当前图包含大量节点。建议：[折叠支线] [只显示主线] [按区域分组] [打开小地图]       |
| 危险操作  | 这会导致主线不可通关。你可以取消、仍然放置，或创建特殊获取机制。                  |

### 18.3 规则预设

| 游戏类型      | 默认规则                                                   |
|-----------|--------------------------------------------------------|
| 传统迷宫      | 死路过多需提示；错误路径最好有奖励或地标；关键路径转向过多需地标。                      |
| Zelda 式地牢 | 小钥匙不能导致错误开门软锁；核心道具获得后至少有多多个使用点；Boss Key 必须早于 Boss 门可达。 |
| 恐怖解谜      | 长回溯应有事件或空间变化；高压段需要缓冲；安全屋距离和追逐路线需验证。                    |

|              |                                          |
|--------------|------------------------------------------|
| 叙事迷宫         | 知识锁所需线索必须在锁前可达；揭示顺序应递进；关键叙事事实不应被放在其用途之后。 |
| Metroidvania | 能力门必须有对应能力；单向落差需出口；能力获得后应改变多个旧区域。        |

### 18.4 研发交接备注

为了便于研发落地，UI 原型应与数据模型绑定。每个可视组件应能映射到项目数据中的 id、type、location、requires、outputs、tags 和 validationState。

```
{
  "spaces": [{"id": "main_hall", "name": "中央大厅"}],
  "connections": [{"from": "main_hall", "to": "basement", "gate": "basement_lock"}],
  "gates": [{"id": "basement_lock", "requires": ["basement_key"]}],
  "tokens": [{"id": "basement_key", "location": "library"}],
  "beats": [{"id": "first_lock", "space": "main_hall", "type": "obstacle"}]
}
```

### 参考与理论基础

本产品概念以 Environment Tree 的“锁住连接即锁住子树、从目标反向放置解法、保证钥匙位于锁外可达区域”的思想为理论基础，并将其扩展为 Space + Gate + Token + Puzzle + Beat 的通用空间谜题模型。

- Environment Tree: <https://www.squidi.net/three/entry.php?id=4>